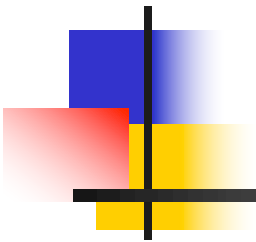
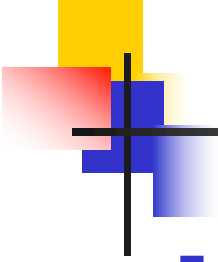


Tema 8: Arquitecturas y Tecnologías Relacionadas



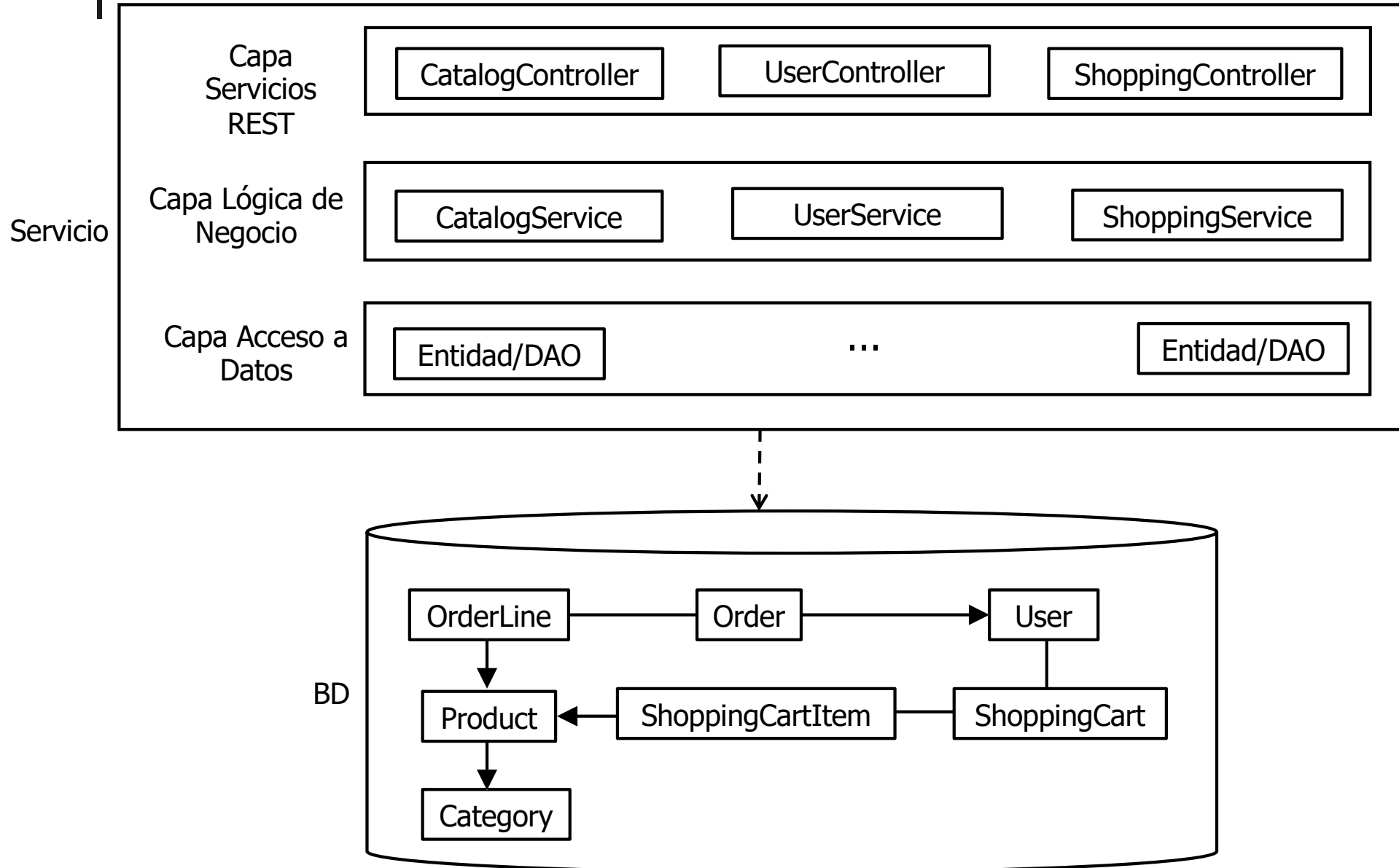


Índice

- Backend
 - Arquitecturas basadas en microservicios
- Frontend
 - Desarrollo de aplicaciones nativas con tecnologías web
- Aplicabilidad de los conocimientos adquiridos

[Backend] Arquitecturas basadas en microservicios

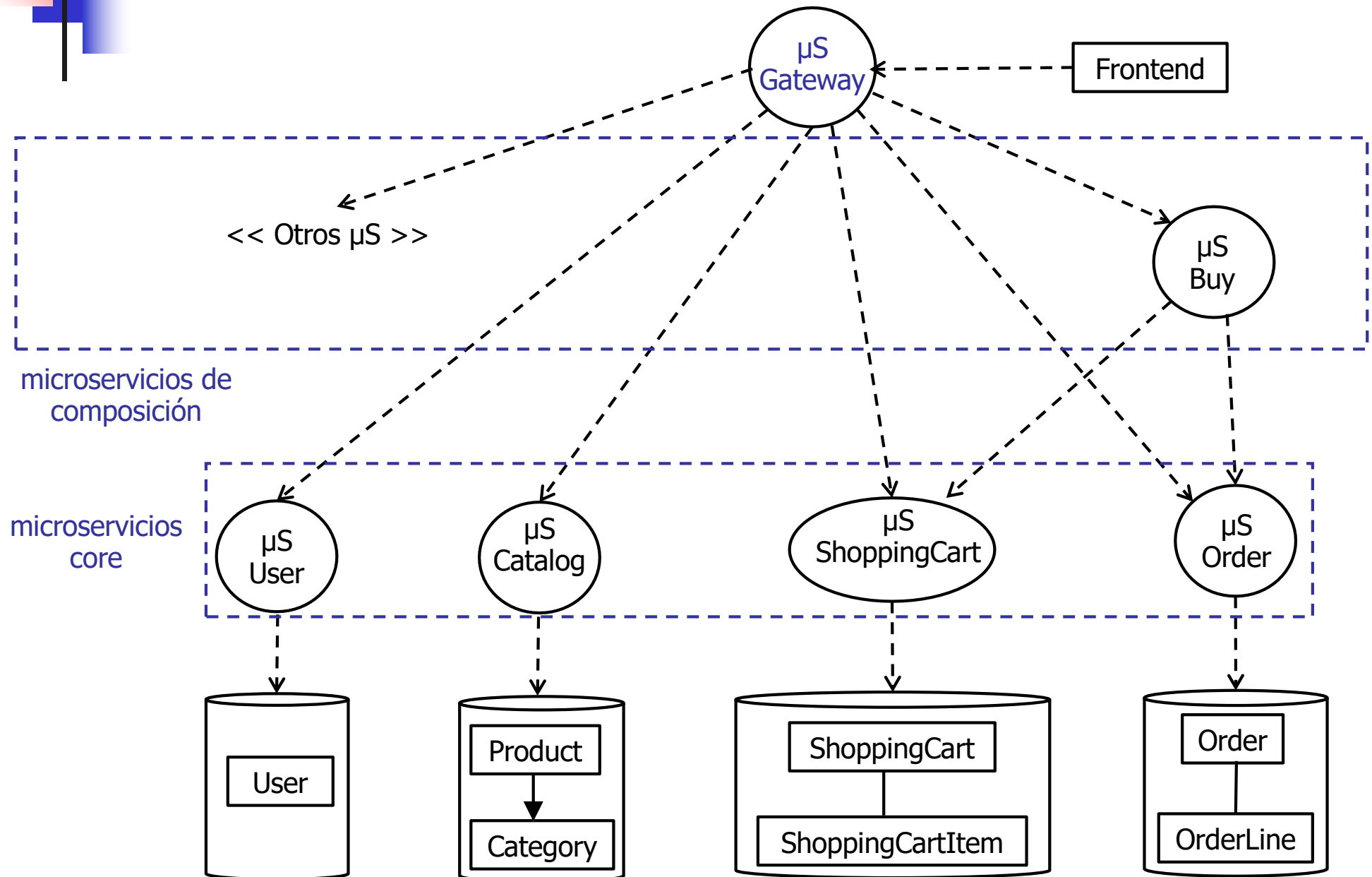
Arquitectura monolítica

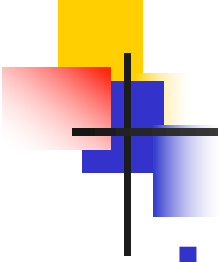


- Todo el backend corre dentro de un servicio
- En la mayoría de los casos esto no representa un problema
- Sin embargo
 - Cuando el código del backend es demasiado grande, puede ser más complejo de gestionar, tanto en desarrollo como en despliegue
 - Un cambio de una línea de código en un backend de un millón de líneas de código, requiere redesplegar todo el backend si queremos que se aplique el cambio
 - Cuando se desea escalar el backend, es necesario replicar el único (gran) servicio múltiples veces
 - Y quizás, las necesidades de escalabilidad de, por ejemplo, búsqueda y compras, sean distintas

[Backend] Arquitecturas basadas en microservicios

Un ejemplo

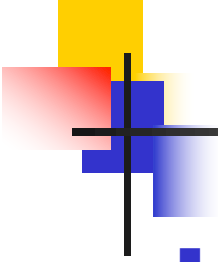




[Backend] Arquitecturas basadas en microservicios

Un ejemplo

- La anterior transparencia ilustra una arquitectura de ejemplo basada en microservicios para pa-shop
 - La funcionalidad del backend se descompone en múltiples servicios (mucho) más pequeños, llamados microservicios
 - Una arquitectura real basada en microservicios, tendría muchos más
- Microservicios “core”
 - Implementan casos de uso finales (e.g. *μS Catalog* proporciona las búsquedas de productos por clave e id) y/o casos de uso internos (e.g. *μS Order* implementa un caso de uso para añadir un pedido)
 - Cada uno tiene su propia BD
- Microservicios de composición
 - Implementan casos de uso que requieren la cooperación de varios microservicios
 - E.g. *μBuy* implementa el caso de uso de comprar: recupera el carrito haciendo uso de *μS Shopping Cart*, crea un pedido haciendo uso de *μS Order* y vacía el carrito haciendo uso de *μS Shopping Cart*
- Microservicio Gateway
 - Actúa como fachada del backend al frontend
 - Delega en los microservicios subyacentes



[Backend] Arquitecturas basadas en microservicios

- Ventajas

- Desarrollo

- Cada microservicio puede ser desarrollado por un equipo diferente (en un proyecto propio)
- Cada microservicio puede desarrollarse en la tecnología que se considere más apropiada y usar el tipo de BD (si la necesita) que se considere más apropiado

- Despliegue

- Cada microservicio puede desplegarse independientemente del resto, es decir, sin necesidad de re-desplegar ningún otro microservicio (incluso los que dependen del él)

- Escalabilidad

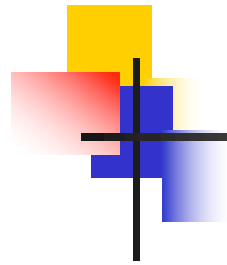
- Cada microservicio puede escalarse de forma independiente, mejorando la escalabilidad del sistema total

- Desventajas

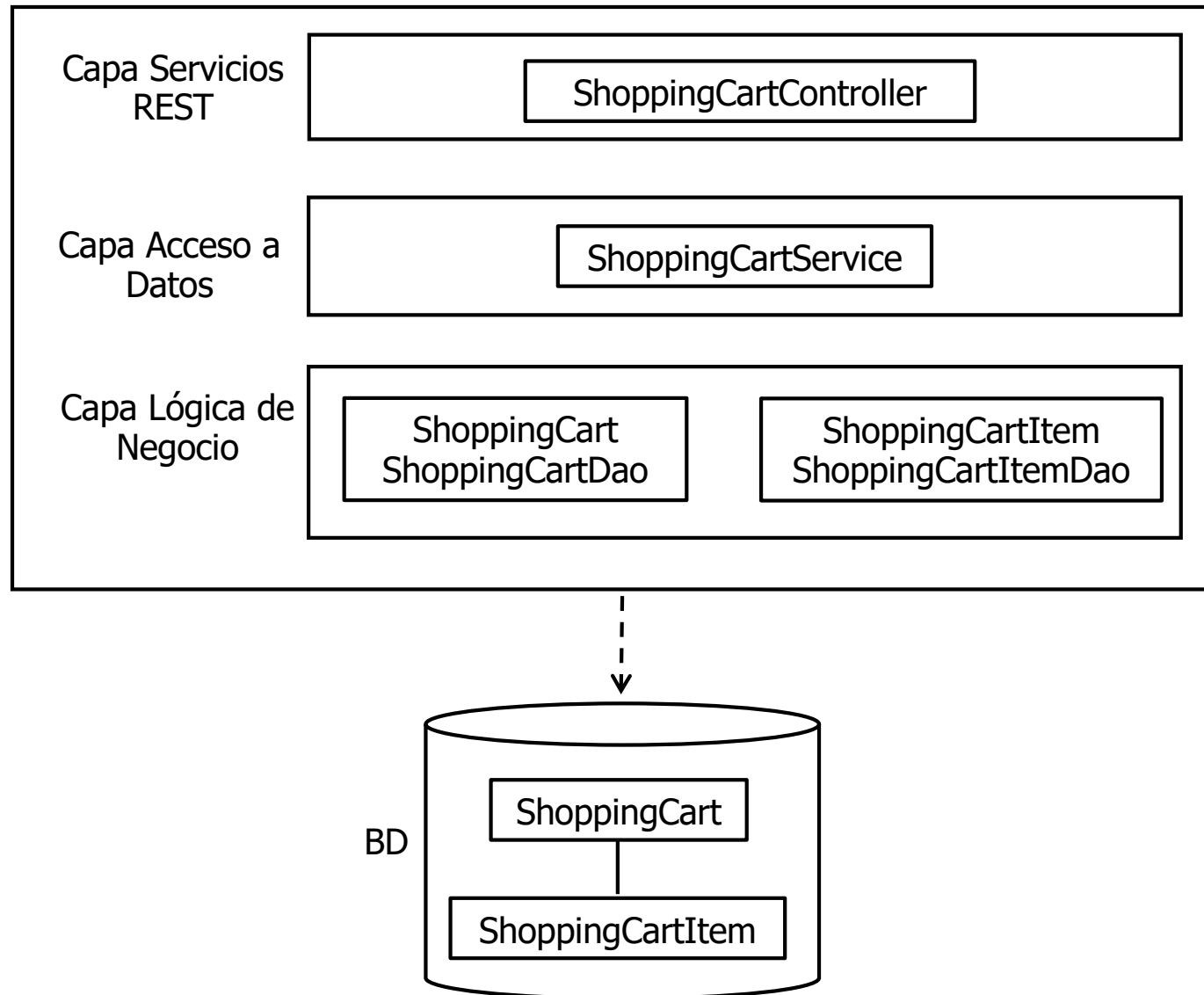
- Una arquitectura mucho más compleja, sólo justificable para backends muy grandes y/o con altas necesidades de escalabilidad

[Backend] Arquitecturas basadas en microservicios

Anatomía de un microservicio (ejemplo)

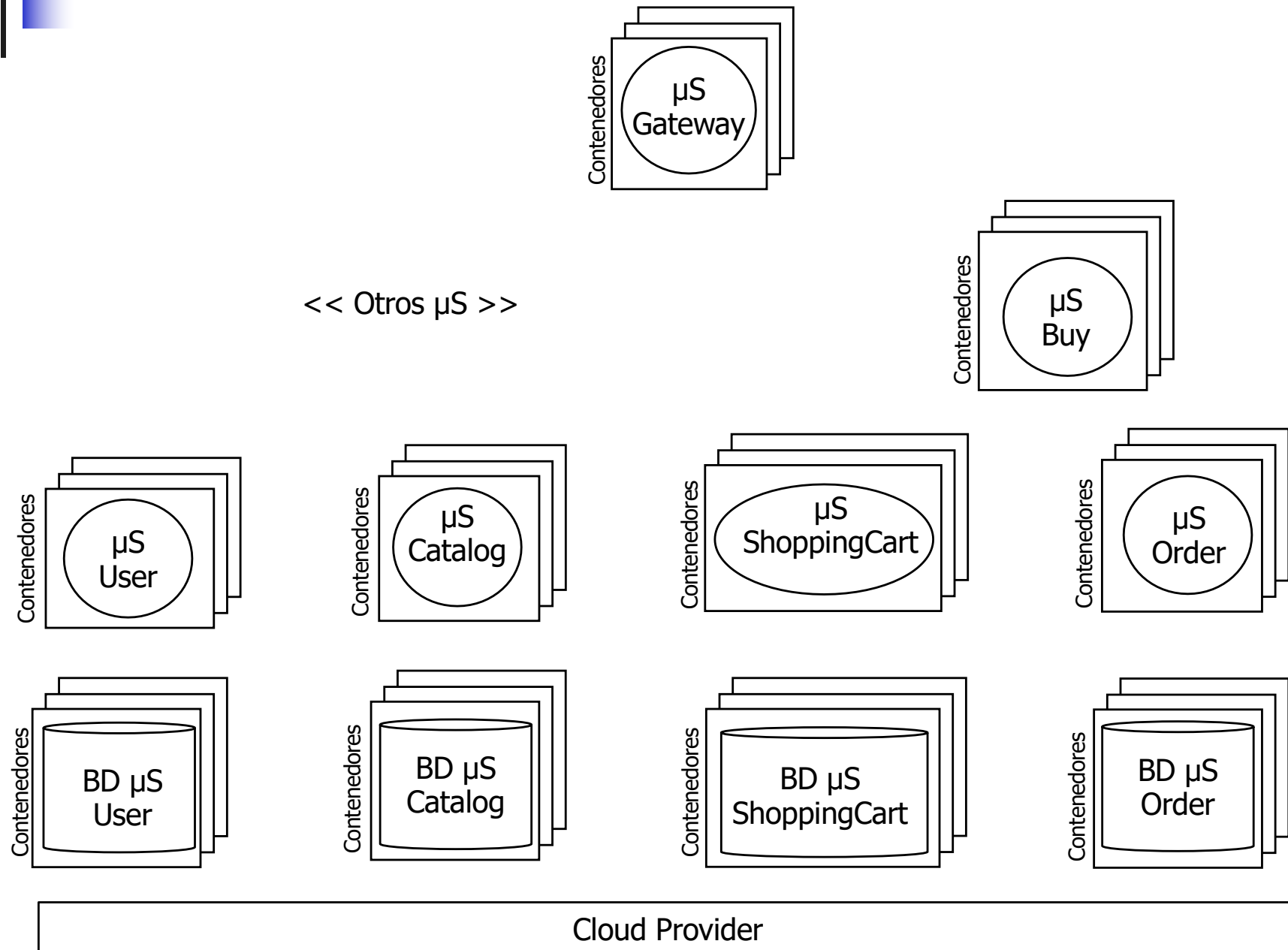


μS
ShoppingCart

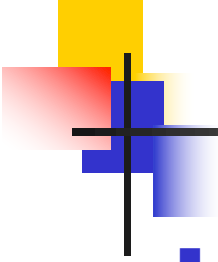


[Backend] Arquitecturas basadas en microservicios

Despliegue



- Por escalabilidad y tolerancia a fallos, cada microservicio está replicado en varias instancias
- Cada instancia de un microservicio o BD corre dentro de un **contenedor**
 - Un contenedor es una abstracción que permite paquetizar un software (e.g. un microservicio) con todas sus dependencias necesarias (e.g. JVM, librerías del sistema, variables del entorno, etc.)
 - Cuando un contenedor se ejecuta, el árbol de procesos que arranca se ejecuta dentro del sistema operativo de la máquina en la que corre el contenedor, y ese árbol de procesos se ejecuta de forma **aislada** del resto de contenedores que también corran en esa máquina
 - La máquina en la que corre un contenedor puede ser una máquina física o una máquina virtual
 - El modelo de contenedores más popular es Docker (<https://www.docker.com>)



[Backend] Arquitecturas basadas en microservicios Despliegue

- Por agilidad/manejabilidad, las arquitecturas basadas en microservicios se despliegan en plataformas cloud
 - Públicas
 - <https://aws.amazon.com>
 - <https://azure.microsoft.com>
 - <https://cloud.google.com>
 - Privadas (“on premise”)
 - <https://www.cloudfoundry.org>
 - <https://www.redhat.com/en/technologies/cloud-computing/openshift>

- Aspectos de diseño/implementación
 - ¿Qué tecnología de comunicación emplear para la comunicación interna entre microservicios?
 - ¿Cómo hacer frente a fallos de comunicación entre microservicios?
 - ¿Cómo una instancia de un microservicio localiza a otra instancia de otro microservicio?
 - ¿Cómo se balancea la carga?
 - ¿Cómo implementar transacciones distribuidas de forma escalable y con qué semántica? (e.g. microservicio Buy)
 - ¿Cómo se accede a la información de configuración?
 - ¿Cómo se securizan los microservicios?
 - Etc.

- Más información

- <https://microservices.io>
- <https://www.f5.com/company/blog/nginx/introduction-to-microservices>
- <https://spring.io/microservices>

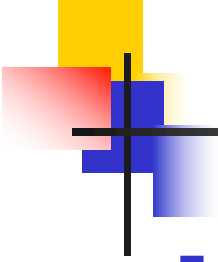
- Los SDKs nativos de los sistemas operativos para desktop y móvil permiten desarrollar aplicaciones nativas con las máximas capacidades posibles
- En contrapartida, requieren desarrollar una aplicación nativa (frontend) distinta para cada sistema operativo que se desee soportar
 - Cada aplicación se desarrolla en un lenguaje concreto y con unas APIs específicas
- Alternativamente, existen soluciones **cross-platform**, que a partir de una única distribución fuente generan ejecutables para distintos sistemas operativos
- Existen varios tipos de soluciones cross-platform
 - [1] Las que compilan a código nativo
 - [2] Las que usan tecnologías web
 - [2.1] Las que permiten implementar aplicaciones híbridas
 - [2.2] React Native

- [1] Soluciones cross-platform que compilan a código nativo
 - Se desarrolla en un lenguaje de programación y se proporciona un compilador que genera un ejecutable para cada plataforma soportada
 - .NET Multi-Platform App UI
 - <https://dotnet.microsoft.com/es-es/apps/maui>
 - C#
 - Flutter
 - <https://flutter.dev>
 - Dart

- [2.1] Soluciones cross-platform que permiten implementar **aplicaciones híbridas**
 - Se implementan igual que una aplicación web SPA
 - Internamente corren dentro un navegador embebido
 - Se puede usar Angular, React, Vue.js, etc.
 - Capacitor: <https://capacitorjs.com>
 - Electron: <https://electronjs.org>

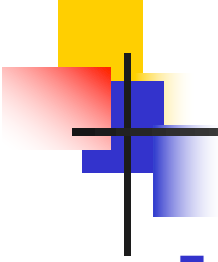
- [2.2] React Native

- <https://reactnative.dev>
- Permite implementar aplicaciones nativas en JavaScript y React
- No usa un navegador embebido
- React Native proporciona componentes predefinidos para los componentes nativos (**View**, **Text**, **Button**, **TextInput**, etc.)
- Mismo modelo de programación que para aplicaciones Web SPA con React, usando esos componentes predefinidos en lugar de los componentes predefinidos para los elementos HTML



Aplicabilidad de los conocimientos adquiridos

- Los conocimientos adquiridos en la asignatura se pueden aplicar directamente al desarrollo de otro tipo de aplicaciones/servicios
 - Backends Java para aplicaciones web SPA implementadas con otros frameworks
 - Backends Java para aplicaciones web del lado servidor (con capa Modelo local o remota)
 - Backends Java para aplicaciones nativas (móviles y desktop)
 - Aplicaciones web SPA con React que usen un Backend no Java
 - Aplicaciones nativas híbridas
 - Aplicaciones nativas con React Native



Aplicabilidad de los conocimientos adquiridos

- E indirectamente

- Los conceptos explicados para el desarrollo del backend aplican también a otro tipo de tecnologías
- Otros frameworks JavaScript para aplicaciones web SPA (e.g. Angular, Vue) también usan el concepto de desarrollo basado en componentes
- La soltura ganada en el desarrollo del frontend en JavaScript hace que el desarrollo de backends en JavaScript (e.g. Node/Express) esté a un paso